



TUGAS 2 TUTORIAL ONLINE 5

NAMA : AKHMAD MUDEHIR
NIM : 055745857
PROGRAM STUDI : SISTEM INFORMASI
FAKULTAS : SAINS DAN TEKNOLOGI
UPBJJ/UT DAERAH : JAKARTA
NAMA/KODE MATA KULIAH : SISTEM OPERASI / STSI4103
KODE KELAS : 38

SOAL TUGAS

1. Sebuah CPU dengan quantum 2 mendapat beberapa proses yang kedatangannya sebagai berikut:

Process	Arrival Time (waktu datang)	Burst Time
P1	0	5
P2	1	4
P3	3	3
P4	5	1

Hitung lah rata-rata dari Turnaround time, Waiting Time dan Response time menggunakan algoritma penjadwalan Round Robin

2. Hitung lah rata-rata dari **Turnaround time**, **Waiting Time** dan **Response time** menggunakan algoritma penjadwalan Round Robin

- Terdapat dua siswa, sebut: **Budi** dan **Andi** yang masing-masing membutuhkan 2 buah alat tulis yang sama, yaitu: **Pensil** dan **penghapus**.
- Budi meminta ijin untuk menggunakan Pensil terlebih dahulu. Karena Pensil memang sedang tidak ada yang menggunakan, maka Pensil diberikan kepada Budi.
- Pada waktu yang hampir bersamaan, Andi meminta ijin untuk menggunakan Penghapus. Karena Penghapus memang sedang tidak ada yang menggunakan, maka Penghapus diberikan kepada Andi.
- Setelah Budi mendapatkan Pensil, langkah selanjutnya adalah meminta ijin untuk menggunakan Penghapus. Karena Penghapus sedang digunakan oleh Andi, maka Budi menunggu sampai Penghapus tidak digunakan.
- Disisi lain, Andi sudah mendapatkan Penghapus, yang kemudian meminta ijin untuk menggunakan Pensil. Karena Pensil sedang digunakan oleh Budi, maka Andi menunggu sampai Pensil tidak digunakan.
- Situasinya akan menjadi Budi menunggu Andi melepaskan Penghapus, padahal Penghapus akan dilepaskan jika Andi sudah mendapatkan Pensil (yang sedang digunakan Budi) dan menyelesaikan prosesnya. Begitu juga sebaliknya.

Dari **permasalahan** proses diatas, bagaimana cara **melakukan proses pemberhentian** pada kasus proses di atas!

3. Memori merupakan komponen terpenting pada komputer modern saat ini, memori menyimpan program-program yang akan di eksekusi oleh CPU. Namun dengan semakin banyak program-program dan ukuran program semakin besar juga menimbulkan ukuran memori fisik lebih kecil dari pada kebutuhan program yang akan dieksekusi oleh CPU. **Metode atau konsep** apa yang cocok digunakan untuk **menyelesaikan permasalahan** tersebut? Jelaskan juga **teknik dari konsep** yang anda gunakan?

JAWABAN TUGAS

1. Analisis Perhitungan Algoritma Round Robin (Quantum = 2)

Untuk menghitung rata-rata Turnaround Time (TAT), Waiting Time (WT), dan Response Time (RT), kita perlu membuat Gantt Chart terlebih dahulu untuk memvisualisasikan eksekusi proses.

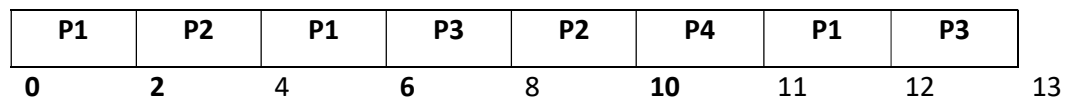
Rumus perhitungan yang digunakan:

- **Completion Time (CT):** Waktu saat sebuah proses selesai dieksekusi (dilihat dari Gantt chart).
- **Turnaround Time (TAT):** Total waktu proses berada di sistem.
 - $TAT = CT - Arrival\ Time\ (AT)$
- **Waiting Time (WT):** Total waktu proses menunggu di antrian (tidak dieksekusi).
 - $WT = TAT - Burst\ Time\ (BT)$
- **Response Time (RT):** Waktu dari proses datang (AT) hingga pertama kali dieksekusi.
 - $RT = WaktuEksekusiPertama\ (WEP) - Arrival\ Time\ (AT)$

Data Proses:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	4
P3	3	3
P4	5	1

Quantum (q): 2

Grafik Gantt:**Penjelasan singkat alur Data Proses dan Grafik Gantt:**

Waktu	Proses	Antrian
Waktu 0	P1 datang, sehingga P1 yang dijalankan	P1
Waktu 0-2	P1 di eksekusi ($q=2$), sisa BT P1=3 ($5-2 = 3$)	
Waktu 2	P2 datang. Masuk ke antrian (pada Waktu=1). Quantum P1 habis, P1 kembali ke antrian	P2,P1
Waktu 2-4	P2 di eksekusi ($q=2$), sisa BT P2=2 ($4-2 = 2$)	
Waktu 4	P3 datang. Masuk ke antrian (pada Waktu=3). Quantum P2 habis, P2 kembali ke antrian	P1,P3,P2
Waktu 4-6	P1 di eksekusi ($q=2$), sisa BT P1=1 ($3-2 = 1$)	
Waktu 6	P4 datang. Masuk ke antrian (pada Waktu=5). Quantum P1 habis, P1 kembali ke antrian	P3,P2,P4,P1
Waktu 6-8	P3 di eksekusi ($q=2$), sisa BT P3=1 ($3-2 = 1$)	
Waktu 8	Quantum P3 habis, P3 kembali ke antrian	P2,P4,P1,P3
Waktu 8-10	P2 di eksekusi ($q=2$), sisa BT P2=0 ($2-2 = 0$, habis). P2 Selesai pada 10 ms)	
Waktu 10		P4,P1,P3
Waktu 10-11	P4 di eksekusi ($q=2$), karena $BT < q$ maka sisa BT P4=0 ($1-1 = 0$, habis). P4 Selesai pada 11 ms)	
Waktu 11		P1,P3
Waktu 11-12	P1 di eksekusi ($q=2$), karena $BT < q$ maka sisa BT P1=0 ($1-1 = 0$, habis). P1 Selesai pada 12 ms)	
Waktu 12		P3
Waktu 12-13	P3 di eksekusi ($q=2$), karena $BT < q$ maka sisa BT P3=0 ($1-1 = 0$, habis). P3 Selesai pada 13 ms)	

Data yang telah didapat:

- P1 selesai dalam waktu **12 ms**, Waktu mulai pada **0 ms**
- P2 selesai dalam waktu **10 ms**, Waktu mulai pada **2 ms**
- P3 selesai dalam waktu **13 ms**, Waktu mulai pada **6 ms**
- P4 selesai dalam waktu **11 ms**, Waktu mulai pada **10 ms**

Tabel Perhitungan Keseluruhan:

Proses	AT	BT	CT	TAT (CT – AT)	WT (TAT – BT)	RT (WEP – AT)
P1	0	5	12	$12 - 0 = 12$	$12 - 5 = 7$	$0 - 0 = 0$
P2	1	4	10	$10 - 1 = 9$	$9 - 4 = 5$	$2 - 1 = 1$
P3	3	3	13	$13 - 3 = 10$	$10 - 3 = 7$	$6 - 3 = 3$
P4	5	1	11	$11 - 5 = 6$	$6 - 1 = 5$	$10 - 5 = 5$
Jumlah				37	24	9
Rata-rata				9.25	6	2.25

Hasil Rata-Rata:

- **Rata-rata Turnaround Time (Avg TAT):** Total TAT / Jumlah Proses = $37 / 4 = 9.25$
- **Rata-rata Waiting Time (Avg WT):** Total WT / Jumlah Proses = $24 / 4 = 6$
- **Rata-rata Response Time (Avg RT):** Total RT / Jumlah Proses = $9 / 4 = 2.25$

2. Analisis Permasalahan Proses (Deadlock)

Kasus yang digambarkan antara Budi dan Andi adalah contoh penerapan dari **Deadlock (Kebuntuan)**.

Situasi ini memenuhi **empat syarat terjadinya deadlock**:

1. **Mutual Exclusion:** Satu sumber daya (Pensil/Penghapus) hanya dapat digunakan oleh satu proses (Budi/Andi) pada satu waktu.
2. **Hold and Wait:** Budi menahan Pensil sambil menunggu Penghapus. Andi menahan Penghapus sambil menunggu Pensil.

3. **No Preemption:** Sumber daya tidak dapat diambil paksa. Pensil tidak bisa diambil dari Budi, Penghapus tidak bisa diambil dari Andi.
4. **Circular Wait:** Budi menunggu Andi (untuk Penghapus), dan Andi menunggu Budi (untuk Pensil), membentuk siklus saling tunggu.

Cara Melakukan Pemberhentian (Resolusi Deadlock):

Karena sistem sudah masuk dalam kondisi deadlock, "pemberhentian" atau resolusi harus dilakukan untuk memulihkan sistem. **Metode yang paling umum** adalah:

1. **Pemberhentian Proses (Process Termination)** Metode ini adalah yang paling langsung untuk memutus siklus tunggu (Circular Wait) dengan menghentikan satu atau lebih proses yang terlibat.
 - **Menghentikan Semua Proses:** Sistem dapat langsung menghentikan Budi dan Andi. Ini adalah solusi "brute force" yang pasti berhasil memutus deadlock, tetapi biayanya mahal. Dalam komputasi, ini ibarat melakukan force quit pada kedua aplikasi yang macet.
 - **Menghentikan Satu Proses (Selektif):** Metode yang lebih baik adalah sistem memilih satu "korban" untuk dihentikan. Misalnya, sistem operasi memutuskan untuk menghentikan proses Budi.
 - Ketika proses Budi dihentikan, dan semua sumber daya yang dipegangnya (Pensil) dilepaskan.
 - Maka setelah Pensil dilepas, sumber daya itu dapat dialokasikan ke Andi. Andi, yang sudah memegang Penghapus, sekarang mendapatkan Pensil. Andi dapat menyelesaikan pekerjaannya dan kemudian melepaskan kedua sumber daya (Pensil dan Penghapus).
 - Sehingga setelah Andi selesai, sistem dapat me-restart proses Budi dari awal, yang sekarang dapat memperoleh Pensil dan Penghapus tanpa halangan.

2. **Preemption Sumber Daya (Resource Preemption)** Metode ini melibatkan pengambilan paksa sumber daya dari satu proses dan memberikannya ke proses lain.

- Sistem operasi memaksa Budi untuk melepaskan Pensil (meskipun Budi belum selesai). Ini disebut preempting sumber daya.
- Pensil tersebut kemudian diberikan kepada Andi. Andi sekarang memiliki Pensil dan Penghapus, menyelesaikan pekerjaannya, dan melepaskan keduanya. Setelah itu, sumber daya dikembalikan ke Budi (bersama dengan Penghapus) agar Budi dapat melanjutkan.
- Metode ini kompleks karena sistem harus bisa mengembalikan proses "korban" (Budi) ke keadaan aman sebelumnya (rollback), yang tidak selalu mungkin dilakukan.

Pada kasus Budi dan Andi, Pemberhentian Proses Selektif (menghentikan salah satu, misal Budi) adalah cara paling jelas dan umum untuk "memberhentikan" kebuntuan tersebut.

3. Analisis Konsep Mengatasi Keterbatasan Memori Fisik (RAM)

Dalam kasus ini, terdapat permasalahan yang terjadi di mana ukuran program (baik satu program besar atau banyak program kecil) melebihi kapasitas memori fisik (RAM) adalah masalah fundamental dalam komputasi modern.

Konsep yang digunakan: Memori Virtual (Virtual Memory)

Konsep yang paling cocok untuk menyelesaikan permasalahan ini adalah **Memori Virtual**.

Kenapa cocok? Karena memori virtual adalah sebuah teknik manajemen memori yang memberikan ilusi kepada setiap program (proses) bahwa ia memiliki ruang memori yang sangat besar (ruang alamat virtual) dan bersifat privat (tidak diganggu proses lain), padahal kenyataannya memori fisik (RAM) yang tersedia terbatas dan harus dibagi-bagi.

Cara kerjanya? Inti dari memori virtual adalah memisahkan antara alamat memori logis (yang dilihat oleh program) dan alamat memori fisik (yang ada di chip RAM). Sistem operasi, dibantu oleh perangkat keras khusus (MMU/Memory Management Unit), mengelola pemetaan ini.

Teknik implementasi: Paging (Penomoran Halaman)

Teknik yang paling umum untuk menerapkan memori virtual adalah **Paging**.

1. **Pembagian:** Ruang alamat virtual (milik program) dibagi menjadi blok-blok berukuran tetap yang disebut Halaman (Pages). Memori fisik (RAM) juga dibagi menjadi blok-blok berukuran sama yang disebut Bingkai (Frames).
2. **Pemuatan Sebagian (Partial Loading):** Kunci utamanya adalah: Tidak semua bagian program perlu ada di RAM secara bersamaan. Saat program dijalankan, OS hanya memuat sebagian kecil halaman program (yang benar-benar dibutuhkan saat itu, misal fungsi `main()`) ke dalam beberapa bingkai di RAM.
3. **Penyimpanan Sekunder:** Sisa halaman program lainnya (yang belum diperlukan) disimpan di penyimpanan sekunder (seperti SSD atau HDD) dalam sebuah file khusus yang disebut page file atau swap space.
4. **Mekanisme Page Fault:**
 - Ketika CPU mencoba mengakses alamat memori yang halamannya tidak ada di RAM, MMU akan memicu interupsi yang disebut **Page Fault**.
 - Ini bukan error, tapi sinyal untuk OS.
 - OS kemudian mengambil alih, mencari halaman yang dibutuhkan di SSD, mencari bingkai kosong di RAM (atau mengosongkan bingkai yang berisi halaman lama/tidak terpakai), lalu menyalin halaman yang dibutuhkan dari SSD ke RAM.
 - Setelah halaman ada di RAM, OS memperbarui tabel pemetaan (Page Table) dan menginstruksikan CPU untuk mengulangi instruksi yang gagal tadi. Kali ini, instruksi akan berhasil.

Contoh studi kasus nyata:

Kita bisa mulai dengan menjalankan game modern kelas berat seperti Cyberpunk 2077 atau Microsoft Flight Simulator pada laptop dengan RAM fisik 8 GB.

Setelah kita analisis spesifikasinya, ternyata rekomendasi sistem untuk game tersebut seringkali 16 GB atau bahkan 32 GB. Game itu sendiri mungkin memiliki aset (tekstur detail, model 3D, suara) berukuran total 100 GB.

Implementasi menggunakan **Memori Virtual (Virtual Memory)**:

- Sebenarnya mustahil untuk memuat seluruh game ke RAM 8 GB.
- Dengan memori virtual (misalnya, di Windows), OS menggunakan teknik Demand Paging. OS hanya memuat halaman-halaman yang esensial untuk memulai game dan area pertama (misalnya, 6 GB data) ke dalam RAM.
- Sisa data game tetap berada di SSD (misalnya, di file pagefile.sys).
- Ketika player teleportasi ke area kota yang baru, CPU akan meminta data tekstur gedung baru. Data ini ada di halaman yang belum dimuat. Lalu terjadilah Page Fault.
- Windows akan menjeda game, kemudian mencari halaman tekstur lama yang sudah tidak terlihat (misal, area yang baru ditinggalkan), memindahkannya keluar dari RAM ke pagefile.sys di SSD, lalu memuat halaman tekstur baru dari SSD ke bingkai RAM yang baru kosong.
- Proses ini (disebut swapping atau paging) memungkinkan game yang membutuhkan >8 GB data untuk berjalan di sistem 8 GB, meskipun mungkin dengan sedikit stutter (jeda) saat halaman baru dimuat.

Kesimpulan

Untuk memperkuat pemahaman saya, dapat disimpulkan dengan 3 poin sebagai berikut:

1. **Algoritma Round Robin (RR)** efektif untuk sistem time-sharing karena memberikan response time yang baik (terlihat dari Avg RT yang rendah, 2.25) dan mencegah satu proses memonopoli CPU. Namun, RR memiliki waiting time dan turnaround time yang cenderung lebih tinggi (Avg WT 6.0, Avg TAT 9.25) dibandingkan algoritma non-preemptive karena adanya pergantian konteks (context switch) yang sering.
2. **Deadlock** adalah kondisi kritis di mana dua atau lebih proses saling menunggu sumber daya yang dipegang satu sama lain, menciptakan siklus tunggu yang tidak bisa selesai. Resolusi paling umum adalah dengan pemberhentian proses (baik secara paksa atau selektif) untuk memutus siklus tersebut.
3. **Memori Virtual** adalah konsep fundamental yang memungkinkan eksekusi program yang lebih besar dari RAM fisik. Dengan teknik Paging, sistem operasi menciptakan ilusi memori yang besar dengan hanya menyimpan bagian program yang aktif di

RAM dan sisanya di penyimpanan sekunder (SSD/HDD), memuatnya sesuai permintaan melalui mekanisme Page Fault.

Sumber referensi:

- Rinusantoro, S. (2025). Sistem Operasi (MSIM4201). Edisi 2. Modul 4 Hlm 4.27 - 4.28 Tangerang Selatan: Penerbit Universitas Terbuka.
<https://pustaka.ut.ac.id/lib/msim4201-sistem-operasi-edisi-2/>
- Ramadhan, F. (2022). Memahami Virtual Memory. Diakses pada 3 November 2025, dari <https://blog.unnes.ac.id/farhanfreza/2022/11/08/virtual-memory/>
- GeeksforGeeks. (2025). Program for Round Robin Scheduling for the Same Arrival Time. Diakses pada 3 November 2025, dari <https://www.geeksforgeeks.org/operating-systems/program-for-round-robin-scheduling-for-the-same-arrival-time/>
- TutorialsPoint. Introduction of Deadlock in Operating System. Diakses pada 3 November 2025, dari https://www.tutorialspoint.com/operating_system/introduction_to_deadlock_in_operating_system.htm
- GeeksforGeeks. (2025). Difference Between Turn Around Time (TAT) and Waiting Time (WT) in CPU Scheduling. Diakses pada 3 November 2025, dari <https://www.geeksforgeeks.org/operating-systems/difference-between-turn-around-time-tat-and-waiting-time-wt-in-cpu-scheduling/>
- Hoffman, C. (2023). What Is the Windows Page File, and How Do You Configure It?. Diakses pada 3 November 2025, dari <https://www.howtogeek.com/126430/what-is-the-windows-page-file/>